A Modular-Logarithmic Coprocessor Concept

Ilya Osinin
FSUE «Russian Federal Nuclear Center All-Russian Research Institute of Experimental Physics»
Sarov, Russia
iposinin@gmail.com

***Abstract*—The paper presents a conceptual design of an IP module of mathematical coprocessor. It consists of a set of processing cores of the same kind which perform single-cycle scalar, or vector operations with real numbers. The processed data is represented in the modular logarithmic format that provides two levels of translating the original numbers, namely: the modular level instead of the conventional positional system and the logarithmic level instead of the floating point format. Owing to this feature a coprocessor has a higher performance, a higher accuracy and a higher level of reliability, as compared to the known analogs. A prototype coprocessor is an FPGA-based IP module. Companies developing general-purpose processors are the target market for this design.**

Keywords—coprocessor, residue number system (RNS), logarithmic number system (LNS), reconfigurable architecture, highly reliable computing.

I. Introduction

The recent decades demonstrate an exponential growth of computational capabilities and nowadays high-performance computing (HPC) is an important technology for R&D activities in highly developed states all over the world. Besides the speed of computations, we observe a growing interest in the improvement of accuracy and reliability of supercomputers in view of a continuously increasing number of processing cores. For example, the TaihuLight supercomputer, which is the Top-500 leader as of June 2016 [1] has more than 10 million processing cores. Traditionally, floating point operations are used for these purposes in compliance with the standard IEEE-754 [2], which has a number of significant disadvantages. For example, critical round-off errors often lead to incorrect results of computations [3].

On the other hand, the market of commonly used processors does not present devices that allow parallelizing the arithmetic operations themselves at the level of number system. This provides a future potential to increase the speed of computations. It is an urgent problem to develop an IP module of coprocessor, which would supplement the basic on-chip system and extend the existing set of instructions by adding scalar and vector instructions. Such coprocessor could provide additional competitive advantages at the market of supercomputers for developer companies.

This paper considers a concept of an individual IP module of an on-chip system that allows carrying out computations on the basis of modifications to the conventional principles of computing. The product has no direct analogs owing to the use of the modular-logarithmic number system. Such coprocessor implements the scientific and engineering solutions described, in general, in [3].

The present paper has five sections:

- introduction describes the research problem;
- section 2 generalizes results of examining the development relevancy;
- section 3 contains the basic technology description;
- section 4 describes the competitive advantages of the proposed coprocessor and scenarios of its introduction;
- conclusion generalizes results of the project implementation.

II. Examination of the Modular-Logarithmic Computing Relevancy

HPC is one of the most important R&D technologies for the simulation of space, climate, micro-world, macro-world, etc. Results of analysis [3] show that the machine arithmetic format in compliance with the IEEE-754 standard is used everywhere for HPC. Its advantages and disadvantages, as well as the major approaches used to increase the performance and reliability are considered in this section.

A. A method to increase accuracy of computations

IEEE 754 is the widely used standard describing the floating-point number format, in which a real number is represented as a sign, an exponent and a floating-point coefficient (mantissa).

The advantage of such format is in a significantly wider range against values in the fixed-point format. However, it also has significant disadvantages, for example, round-offs in calculations leading to degradation of algebraic properties, such as commutativity and distributivity. As a result, conventional processors often give erroneous results of calculations for the following types of problems [4]:

- ill-conditioned problems;
- problems with coefficients of different scales;
- problems, which are sensitive to equivalent transformations.

To obtain correct solutions to such problems, highly accurate computations are required, which are based on

- programmatic approaches;

978-1-5386-3250-5/17 $31.00 © 2017 IEEE
DOI 10.1109/HPCS.2017.93

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.

- an increased number of bits in the processed real numbers (512 bites and more);
- the use of a logarithmic number system (LNS).

The first two approaches are traditional. Also, it is possible to perform arithmetic operations with fractions without rounding of results (Wolfram Mathematica, MatLab, and some other program packages) [4]. Implementation of a capability to execute such operations in modern processors causes additional time consumption due to their algorithmic complexity and no hardware support in processors and this is a significant disadvantage.

The third approach is a promising alternative to the IEEE-754 format, because LNS does not require matching of exponents. A real number, a is represented by a binary logarithm $L(a)$ of its absolute value and a sign S.

The translation of the given real number from the IEEE-754 format to LNS is performed in the following manner (Fig. 1):

- sign S is taken unchanged;
- the characteristic of the number, E becomes the integral part of the number in LNS, because $\log_2(2^E)=E$;
- the logarithm of mantissa M, which is the fractional part of the number in LNS, is calculated.

The inverse translation is performed similarly, however, the last step is the calculation of the exponentiation with base 2 (antilogarithm) of the fractional part of the given logarithmic number.

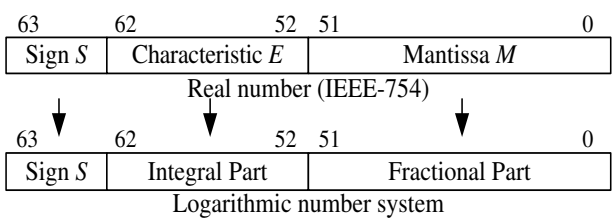

Figure 1. Translation of a double-precision number from the IEEE-754 format to LNS

An important feature of LNS is a simplified execution of the following operations:

- multiplication is replaced by addition $\log_2(a \cdot b)=\log_2 a+\log_2 b$;
- division is replaced by subtraction $\log_2(a/b)=\log_2 a-\log_2 b$;
- exponentiation is replaced by multiplication $\log_2 a^b=b \cdot \log_2 a$;
- extraction of root is replaced by division $\log_2 a^{1/b}=(\log_2 a)/b$;

where a and b are the original operands, log2a and log2b are the logarithmic representations of these operands, respectively.

The advantage provided by the properties of logarithm would allow increasing the speedup of the proposed coprocessor for the category of problems, in which the operations mentioned above dominate over the others. However, the price of such advantage is a more complicated execution of the addition and subtraction operations. For example, addition can be described by the following formula:

$$\log_2(a+b)=\log_2 a+\log_2(1+2^{\log_2 b-\log_2 a}), \quad (1)$$

where a is the smallest operand and b is the largest one. One can see from the formula above that addition consists of a sequence of operations:

- the calculation of antilogarithm of the difference between numbers in LNS;
- adding 1 to the resultant antilogarithm value;
- the calculation of logarithm of the number from the previous step;
- the summation of the obtained logarithm and the least of operands.

Since the point between the integral and fractional parts of the number in LNS is fixed, the matching of exponents is not required and this is a key advantage of this approach in comparison with the floating point format.

The major contribution to studying LNS has been made by J. Coleman, M. Arnold, E. Chester, and D. Lewis [4-6]. However, in spite of a wide scope of research activities we have a few engineering implementations at present time. One of them is the European Logarithmic Microprocessor (ELM), which has a 32-bit RISC architecture [4, 5]. Note, however, that the issue of using data with the number of bits above 32 in LNS is still open. The cause is in the exponentially growing hardware overheads for the code translation from the IEEE-754 format to LNS and vice versa and for the implementation of summation and this is the main disadvantage of such number system.

The engineering solutions described in Section 3, which are based on the research efforts of the author, allow mitigating this disadvantage owing to a higher-order interpolation used for the code translation, which determines the difference between the proposed logarithmic coprocessor and the ELM processor considered above. Besides, the difference is in the use of an additional level of data encoding in a residue number system (RNS). Its specific features are described below.

B. A method to increase performance

Today, the following methods are most popular to increase the speed of computations:

- increase of the number of processing cores in supercomputer, as well as increasing the clock speed of processor (for example, owing to a reduced size of transistors) and due to improvement of algorithms, for example, owing to paralleling of computations;

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.

- increase of the number of instructions per cycle, for example, by using an extended command word in VLIW architecture, or vector operations (up to 8 double-precision instructions per cycle in the Intel AVX coprocessor).

To increase a processing core performance, parallelizing the execution of arithmetic operations at the level of bits of the number system seems to be a promising reserve. In a positional number system (PNS), on which basis the major part of known processors are operated, successive distribution of transfer units prevents such parallelization. Pipelining is the conventional solution to this problem. However, the hardware complexity of such approach significantly increases, as the number of steps increases, and a positive result can be achieved only with a continuous flow of original data.

A promising alternative to it is a modular number system, or, in other words, a residue number system (RNS), in which operations with each digit of number from the set of bases $\{p_1,p_2,...,p_n\}$, where n is the number of residues, are performed independently of each other.

Let RNS consists of two bases $\{p_1=5, p_2=7\}$. The range of numbers is limited by product module. Convert numbers, i.e. find the residues from division by p_1 and p_2. For example, {2,3} is a unique representation of number 17. Residues having a fewer number of bits can be summarized, or multiplied in parallel:

$$+\begin{cases} 17_{10} = \{2_{\bmod 5}, 3_{\bmod 7}\} \\ 10_{10} = \{0_{\bmod 5}, 3_{\bmod 7}\} \end{cases} \qquad (2)$$
$$27_{10} = \overline{\{2_{\bmod 5}, 6_{\bmod 7}\}}$$

The proposed technical solutions on the base of RNS are mainly used for the digital processing of signals, because the use of floating point causes difficulties in view of the difficulties of implementing the division by powers of 2. Scientific works by N. Chervyakov, I. Akushsky, A. Mohan, H. Garner, and A. Omondi have been devoted to this research area [7, 8].

The well-known general-purpose modular processor operating on the basis of artificial neuron networks was developed in Stavropol' in 2005 [7]. However, in scientific papers insufficient attention is paid to a possibility of using the advantages of RNS in combination with the advantages of LNS. This principle is a key difference of the proposed coprocessor from others.

C. A method to improve reliability of computing

Simultaneously with the observed trend of an exponentially growing performance of supercomputers the problem of fault-free computations becomes more and more important, because even a single fault may lead to incorrect results of the many-day computation process. A probability of such faults increases in proportion to the number of processor cores. If a supercomputer includes hundreds of thousands of cores, such faults occur not infrequently.

Besides, there are problems which solution requires the hardware operation reliability in addition to its high performance. Such hardware systems include those operating in real time. In these systems the computing process must be discontinuous. Today, this is achieved owing to

- using backup components of system;
- duplicating, or triplicating computer operations;
- software processing of exception conditions, etc.

A disadvantage of the approaches above consists in that a part of hardware systems is in idle state, or duplicates operations till the time of failure of one of the system components. Besides, the correcting codes (for example, parity checking of processor registers) are used, however, they can only detect a fault in memory reading/writing operations and are not able to detect faults in an arithmetic device.

The different situation arises with the use of a residue number system, which has a natural parallelism potential and, besides, possesses means improving reliability of computations at the level of a number system. In such a case, a single error-control code is used to detect errors occurring during the data processing in arithmetic device and data transmission through communication channels.

The range of P number representation is divided into the working range P_n of n modules and control range P_k of k modules. If any arithmetic operation with two numbers from the working range is executed, the result also belongs to this working range P_n, otherwise an error occurs. The algorithm is described in details in papers [5, 7] by Russian scientists I. Akushsky, I. Kalmykov, and N. Chervyakov, who made a significant contribution to the development of this science area.

III. DESCRIPTION OF THE BASIC TECHNOLOGY

This section describes the proposed modular-logarithmic number system and engineering solutions for a coprocessor which implement arithmetic operations on the basis of such number system.

A fixed point in LNS allows operations with a real operand, as if it is an integer, and this, in turn, allows introducing the next conversion level — conversion to a residue number system (RNS). The result is a new modular-logarithmic number system (MLNS) which combines the advantages of the both previous systems. The number of bits representing any number in the MLNS format may be the same as in the IEEE-754 format providing the equivalent range of real number representation, for example, double precision in case of 64 bits.

The set of bases $P=\{p_1=2^{15}-1,\ p_2=2^{16}-1,\ p_3=2^{16},\ p_4=2^{16}+1\}$ provides a minimum reduction (about 0.003%) of the 63-bit number representation range (the 64-th bit is a sign). The chosen bases meet the requirement of simplicity and allow simplifying the code translation from PNS to RNS and vice versa owing to the modules of the form $2^n\pm1$, where n is the number of bits of module [7].

Direct translation from the IEEE-754 format to MLNS is performed in two stages (Fig. 2):

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.

- the characteristic of the original number with floating point becomes an integral part of logarithm and simultaneously the binary logarithm of the original number's mantissa is calculated; the bit of sign remains unchanged; the point between the integral and fractional parts of the calculated logarithm is dropped out;
- residues from division of the obtained integer by bases of RNS are calculated.

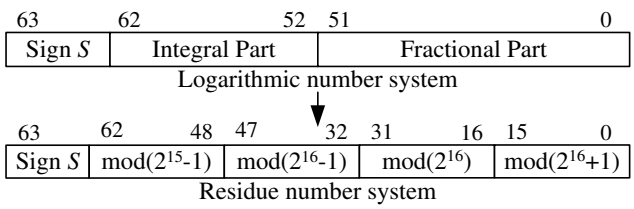

Figure 2. Translation of a double-precision number from LNS to MLNS

The translation from MLNS to the IEEE-754 format is performed in the inversed sequence:

- the known residues are used to calculate the positional representation of logarithm; the point between the integral and fractional parts is reinstated;
- antilogarithm of the fractional part (mantissa) is calculated, the integral part becomes the order of the desired number with floating point; the bit of sign remains unchanged.

It should be noted that with the same number of binary bits in positional and residue number systems (PNS and RNS) the P number representation range in the latter is smaller, because, in general, the n-bit number module is less than 2^n numbers encoded using the same number of bits in a positional number system. Regarding this fact, the choice of a set of bases in a residue number system is the task of a highest priority.

In the proposed concept of coprocessor the translation of numbers from the IEEE-754 format to MLNS and vice versa is performed using a code translator described further in this paper.

A. A code translator

The first stage of encoding a number in MLNS is the calculation of logarithm of its mantissa, which takes values in the interval 1 to 2. The well-known engineering solutions [3,4] use the linear interpolation with additional error correction. This approach provides relatively low hardware overheads and a translation error on the order of $E=10^{-7}$, which is sufficient for single-precision calculations and quite insufficient for the translation of double-precision numbers.

The uniqueness of this solution consists in the use of quadratic interpolation with linear interpolation of error in each subinterval providing a translation error on the order of $E=10^{-16}$, which is sufficient for double-precision calculations. Thus, logarithm y of mantissa x is calculated using the following expression:

$$y = a \cdot x^2 + b \cdot x + c + (al \cdot xl + bl) \cdot maxerr, \tag{3}$$

where a, b, c are interpolation constants for the interval, which the value of x falls into; xl are the least significant 32 bits of mantissa determining a given subinterval; al, bl, $maxerr$ are the subinterval error interpolation constants.

The engineering solution providing the hardware implementation of such number translation technique has the pipeline-type parallelism and includes summators and multipliers. Such solution allows reducing the interpolation error by a factor of billion (from 10^{-7} to 10^{-16}) against the well-known solution with a single-precision and with 8 times higher hardware overheads. The second stage of encoding the given number in MLNS consists in calculating residues of dividing the logarithmic representation of the number by bases of RNS. Owing to the specially selected bases near 2^n the calculation of residues is performed using summators, which have the number of bits not exceeding 16 and this allows significantly reducing the hardware overheads of one module in comparison with the known solutions [5].

Besides the main function of translating codes from the IEEE-754 format to MLNS and vice versa, the hardware translator of codes executes the following operations:

- the calculation of the positional characteristic of a modular number (for example, the representation of a number in a mixed base system [7]), which is required, for example, to compare numbers and detect the word size overflow and computer zero;
- the modular-logarithmic addition and subtraction of real numbers.

The rest arithmetic operations are executed in a processing core.

B. A processing core

Each processing core of coprocessor individually performs the processing of a certain RNS modulus. The multiplication, division and exponentiation operations are executed either in the vector, or scalar form.

In general, the vector length is limited only by hardware costs and data sampling rate. For example, in the considered coprocessor version the length of vector is 128 bits, or 8 line-packed residues belonging to different numbers in MLNS. The choice is determined by matching with vectors, which are processed by the Intel AVX2, where the operation simultaneously with 8 numbers per cycle is carried out.

To process vectors, a processing core has eight (8) 16-bit summators and the same number of multiplicators. The device operates in the following manner. Each cycle, two original vectors from cache-memory arrive at a core for their processing. The following operations, depending on the instruction, are performed with the residue representations of logarithms:

- addition (corresponds to multiplication of the original real numbers);
- subtraction (corresponds to division);

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.

- multiplication (corresponds to exponentiation).

The resultant vector is written to cache-memory. To prevent delays due to extracting and writing data, it is reasonable to use a three-port on-chip cache-memory. Such memory can read two lines and write one line per cycle.

Thus, in contrast to arithmetic logic devices of the well-known general-purpose processors the described processing cores can parallelize the macro-level calculations of operands comprising a vector and micro-level calculations of residues of each operand owing to a small number of bits in the processed residues and the regular structure of processing core.

Arithmetic operations in a processing core are performed within a single cycle without pipelining.

C. *A reconfigurable architecture of coprocessor*

The proposed RNS has four bases and, therefore, a coprocessor consists of four independent cores connected to a code translator and memory (Fig. 3). Cache-memory of each core stores data on the residue, which is processed, rather than the entire number. This allows managing independent calculations in each core and the incoherent cache-memory.

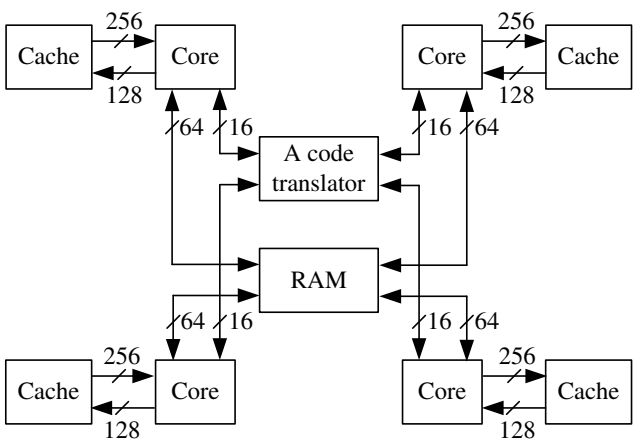

Figure 3. A structural diagram of a modular-logarithmic coprocessor

A key feature of such coprocessor is a possibility of dynamically reconfiguring that allows controlling the number of bits in calculations. This may be important for problems, which don't require high accuracy, but have a high-degree vector parallelism.

The following coprocessor reconfiguration options are possible:

- 32 half-precision vector operations;
- 16 single-precision vector operations;
- 8 double-precision vector operations.

If n coprocessors are located on a chip, the integers processed have maximum $64 \cdot n$ bits. For example, with 8 coprocessors within a single system the integer processing range includes up to 512 bits, with each residue being processed in parallel.

In addition to a possibility of varying the number of bits, reconfiguration allows disabling one, or several cores in case of failure. The way of detecting a single fault, or a double fault in coprocessor, as well as a core failure is based on the RNS properties considered in subsection 2.3. For this purpose, it is necessary to add a control core to coprocessor. In this case the number representation range is divided into the working range and the prohibited range in contrast to general-purpose processors, which require duplicating/triplicating the hardware components, or the process of calculations.

It has been proved [5, 7] that in case of a fault the result obtained falls into the prohibited range and a mechanism of localizing a faulty core has been provided for. If faults continuously occur in one and the same core, the core failure event onsets and the core is locked. In such situation coprocessor continues operating with a reduced range of number representation. If the computation accuracy must not be lowered, it is possible to use processing cores from another coprocessor, including control cores. Such reconfiguration allows creating a processor, which is resistant to gradual degradation.

IV. Results of Testing a Prototype Coprocessor

Currently, the modular-logarithmic processor exists in the form of a prototype IP module based on a field programmable gate array (FPGA). Such choice is justified by simplicity of its implementation, because ready-to-use functional units of soft-processor Altera NIOS can be used, this allows supplementing the basic system with coprocessor instructions and using an existing compiler to write and debug programs. The processor functional diagrams were designed in the Altera's CAD system Quartus II Web Edition. Program debugging and testing were performed on the basis of the Altera Cyclone V card with FPLD.

The hardware costs of the proposed IP module are determined by the use 2 mln transistors, while the implementation of AVX2 of the Intel Xeon (Haswell) processor requires a next-higher order number of 21 mln transistors. Below, the comparison is made with respect to several parameters, such as speed, accuracy and reliability of computing.

A. *Speed of computing*

Fig. 4 illustrates the comparison between the speed of the modular-logarithmic coprocessor (MLC) and the Intel AVX2, with the assumed equal frequency 2.6 GHz of the both devices. In this fig. arithmetic operations with double-precision numbers packed in vectors of 8 elements are designated in the following manner:

- SUM means the summation of the corresponding pairs of vector operands;
- HSUM means the reduction (multi-operand) summation of vector elements;
- MUL/DIV means multiplication/division of pairs of vector operands, respectively;

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.

- POW means that the first vector operands are raised to the power, which is stored in the second vector;
- $\log_2 x$ is a binary logarithm of each vector element;
- matmul means the multiplication of dense matrices of 8*8 elements (it consists of 64 multiplication (MUL) operations and 64 HSUM operations);
- polinom means the calculation of an octic polynomial (it consists of one POW operation, one MUL operation and one HSUM operation).

The operation execution time in a general-purpose processor was measured using the clock() function from the time.h library. To provide the guaranteed execution of vector instructions, the Intel corporation used functions from the set of arithmetic operations of the immintrin.h library.

The highest speedup of the proposed coprocessor is achieved with operations using the properties of logarithms. For example, for the raise-to-power operations the difference in speedup is three orders of magnitude to the advantage of the proposed coprocessor, this is because the Intel coprocessor has no hardware implementation of these operations.

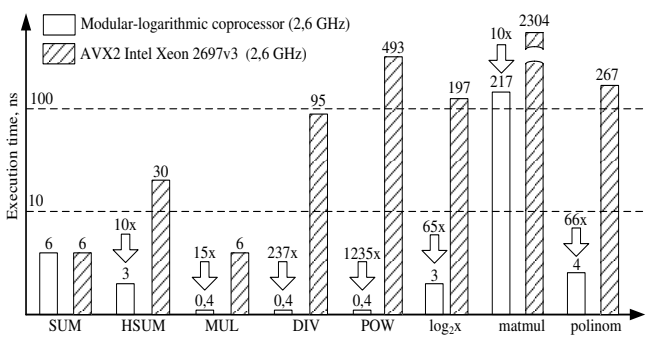

Figure 4. Results of testing for computing speed

The single-cycle execution of vector operations - multiplication, division, and exponentiation - allows achieving a significant advantage in speed for a certain class of problems, where such operations constitute a majority. These operations include the calculation of the n-power polynomials, for example, expansion in Taylor series of matrices and some other operations.

By convention, the translation of numbers to MLNS and vice versa in such testing experiments is performed using the hardware pipelining and, therefore, causes no delays in the computing process.

B. Accuracy of calculations

One of the calculation accuracy testing options is the Cauchy problem solution using the Euler method. Consider a differential equation of the form $x'(t)=t$, where $x_0=0$, $t_0=0$. This equation has the known analytical solution and, hence, it is possible to calculate a relative error of the solution numerically obtained in the floating-point arithmetic. The integration step equals $1/2^i$, where i is a natural number.

Results of double-precision calculations in MLC and in the IEEE-754 format are illustrated by fig. 5, which represents a relative error in the Cauchy problem solution as an integration step function.

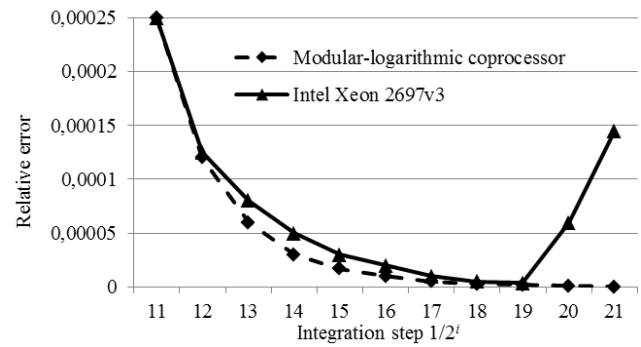

Figure 5. Results of testing for accuracy

One can see from fig. 5 that the relative error of calculations in the IEEE-754 format decreases with a decreasing integration step, however, after the integration step equal to 19 it sharply increases due to round-off errors. This effect is not observed in calculations using a modular-logarithmic coprocessor one can conclude that MLC provides a higher accuracy of calculations in this case.

C. Reliability assessment

The development of reliability characteristics assumes that:

- at initial time a coprocessor has $n=k+r$ cores, where $k=3$ is the number of data cores, r=1 is the number of control cores;
- a minimum number of channels for a functioning coprocessor is $n-(r+1)=2$;
- operability of failed channels is not restored;
- failures of channels are statistically independent events.

Thus, the reliability structure of MLC corresponds to the moving redundancy model, where redundant components are in a loaded state. The probability of operation with no failures is calculated [8] as

$$R_{MLNS}=\sum_{0}^{3\cdot k+4\cdot r} P^{4\cdot(k+r)-i}\cdot(1-P)^i, \tag{4}$$

where $P(t)=e^{-\lambda\cdot t}$ is the probability of no-failure operation of a channel, λ is the rate of failures of a given channel.

In conventional systems of a high reliability level the identification and real-time correction of errors is possible, if only several devices are simultaneously operated on the principle of election, “2 of 3” (quorum). The probability of no-failure operation [8] of such a system is

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.

$$R_{PNS} = 3 \cdot P_{PNS}^2 - 2 \cdot P_{PNS}^3, \quad (5)$$

where $P_{PNS}(t) = e^{-n \cdot \lambda_{PNS} \cdot t}$, λ_{PNS} is the rate of one bit fault, n is the number of bits of a positional processor.

Fig. 6 graphically represents the reliability assessment results for the two versions with the rate of failures λ=10^{-5} h^{-1}. Studying the obtained curves clearly shows the MLC advantages from viewpoint of its reliability. For example, the probability of no-failure operation of MLC for t=2000 hours is R=0,5, while the positional processor's probability of no-failure operation for the same time period is R=0,1. The hardware redundancy of the two processors is 25% and 66%, respectively.

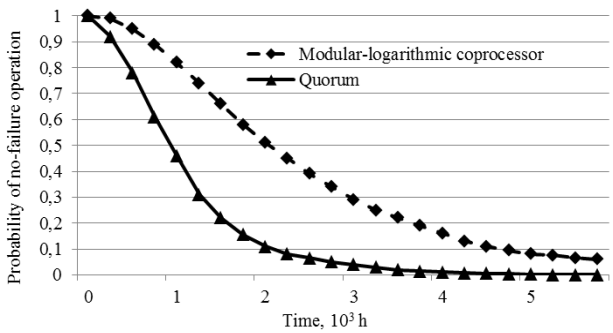

Figure 6. Results of testing for reliability assessment

CONCLUSIONS

The analysis of the approaches used to improve performance, accuracy and reliability of computing and considered in the given paper allowed us to reveal the problems that prevent further improvement of the coprocessor characteristics without fundamental modifications to the calculation principles themselves.

The modular number system instead of the positional system and the logarithmic number system instead of the floating-point format in compliance with IEEE-754 standard seem to be promising variants of systems to implement the required modifications. Thus, an urgent problem is to develop a computing device functioning on the basis of a combined, modular-logarithmic number system.

The R&D efforts resulted in new scientific and engineering solutions that implement the proposed data calculation and encoding schemes and allow achieving the following competitive advantages:

1) A higher speed of computing in comparison with the known analogs owing to:

a) no transfers between moduli of a number;

b) the replacement of multiplication/division operations by addition/subtraction operations;

c) the replacement of exponentiation/ root extraction operations by multiplication/division operations.

For example, the calculation of an arbitrary octic polynomial in a prototype FPGA-based coprocessor is carried out three times quicker in comparison with the Intel Xeon 2697 v3 coprocessor (90 ns against 267 ns), with a 26 times higher clock frequency of the Intel processor (0.1 GHz against 2.6 GHz);

2) A higher accuracy is achieved due to the lack of:

a) rounding of multiplication, division, and exponentiation results;

b) matching of exponents, which leads to the loss of significant digits. This allows correctly solving a number of problems, which are sensitive to round-off errors in the operations mentioned above and contain different-scale data, in contrast to general-purpose processors.

3) In contrast to the well-known computing devices, a higher-level reliability is provided by hardware owing to:

a) a possibility of detecting single-precision and double-precision errors in a computing device;

b) a possibility to reconfigure coprocessor cores in case of failure of one, or several cores and continue the computation process.

It should be noted that the translation of codes from MLNS and vice versa does not cause significant delays with the use of large-size input flows of data owing to the described hardware solutions, which provide pipelining of the process of interpolating the logarithm function and translating codes of a residue number system.

REFERENCES

[1] "Top 10 Sites for November 2016," http://www.top500.org/lists/2016/11 accessed 26/03/2017.

[2] "754-2008 – IEEE Standard for floating-Point Arithmetic," http://ieeexplore.ieee.org/document/4610935/, accessed 26/03/2017.

[3] I. Osinin, "The highly-modular logarithmic processor with a reconfigurable architecture," in *CEUR Workshop Proceedings of the 10th Annual International Scientific Conference on Parallel Computing Technologies (PCT 2016)*, vol. 1576, 2016, pp. 642–654. [Online]. Available: http://ceur-ws.org/Vol-1576/010.pdf

[4] J. Coleman, C. Softley, "The European Logarithmic Microprocessor," in *IEEE Transactions on Computers*, vol. 57, no. 4, pp. 532–546. [Online]. Available: https://doi.org/10.1109/TC.2007.70791

[5] J. Coleman, I. Chester E., "Arithmetic on the European Logarithmic Microprocessor," in *IEEE Transactions on Computers,* vol. 49, no. 7, pp. 702–715. [Online]. Available: https://doi.org/10.1109/12.863040

[6] R. Ismail, R. Hussin, S. Muzad, "Interpolator Algorithms for Approximating the LNS Addition and Substraction," in *IEEE International Conference on Circuits and Systems (ICCAS)*, Kaula Lumpur, Malaysia, Oct 3–4, 2012, pp. 174–179. [Online]. Available: https://doi.org/10.1109/ICCircuitsAndSystems.2012.6408336

[7] A. Mohan Residue Number Systems: Algorithms and Architectures. Bangalore, India: Kluwer Academic Publishers, 2002.

[8] A. Omondi, Residue Number System: Theory and Implementation. London: Imperial College Press, 2007.

Authorized licensed use limited to: Politecnico di Torino. Downloaded on November 06,2024 at 20:07:05 UTC from IEEE Xplore. Restrictions apply.